

PROTOTYPE DESIGN PATTERN

1. THE PROBLEM

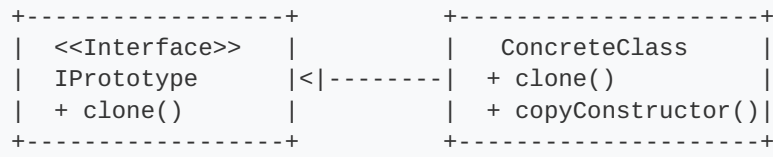
When creating objects requires highly expensive operations like establishing a database connection, performing complex mathematical calculations, or loading heavy files, using the standard new keyword and constructor repeatedly wastes significant time and memory. This can freeze or crash systems, such as when a video game tries to spawn 100 Non-Player Characters (NPCs) that each take 5 seconds to load from scratch.

2. Intuition: Think of making a highly complex origami swan. It takes you an hour to carefully fold the paper just right to create the first one. If someone asks you for 100 swans, it would take you 100 hours to make them all from scratch. Instead, you take your first perfected swan and place it in a 3D scanner/printer, instantly churning out 99 perfect carbon copies in seconds. You only did the hard work once; the rest were just clones.

3. KEY COMPONENTS

COMPONENT	ROLE	EXAMPLE
Prototype Interface	Acts as a marker and declares the clone() method.	IPrototype
Concrete Prototype	Implements the interface, provides a copy constructor, and overrides clone() to return a replica of itself.	NPC
Client	Holds a reference to a Prototype and calls its clone() method to get new objects.	main() application code

4. UML CLASS DIAGRAM



5. HOW IT WORKS — STEP BY STEP

1. Client calls the standard, expensive constructor once to create the base ConcretePrototype object (e.g., `new NPC()`).
2. Client calls the `clone()` method on that base object whenever a new instance is needed.
3. Inside the `clone()` method, the ConcretePrototype calls its own internal copy constructor, passing itself (`this`) as the reference.
4. **Result:** The copy constructor duplicates the internal values into a new memory space and returns a fully formed replica to the Client instantly, completely bypassing the heavy logic of the standard constructor.

6. SHALLOW COPY VS DEEP COPY

ASPECT	SHALLOW COPY	DEEP COPY
What is copied	Primitive data types directly.	Primitive types AND explicitly allocates new heap memory for internal custom objects.
Risk	If used on custom objects, it only copies pointers. Altering the clone alters the original.	Slower to execute, requires careful memory management.

When to use	When your class contains only basic data types (int, string, bool).	When your class holds references or pointers to other objects.
Code example	<code>this.x = other.x</code>	<code>this.obj = new Obj(other.obj)</code>

7. CODE SKELETON (C++)

```
#include <iostream>
#include <string>

// Interface
class IPrototype {
public:
    virtual IPrototype* clone() = 0;
    virtual ~IPrototype() = default;
};

// Concrete class with copy constructor
class NPC : public IPrototype {
private:
    std::string type;
    int health;
    int damage;

public:
    // Standard Expensive Constructor
    NPC(std::string t, int h, int d) : type(t), health(h), damage(d) {
        std::cout << "Running expensive DB query to build base " << type
        << "...\\n";
    }

    // Copy Constructor (Shallow Copy for primitives)
    NPC(const NPC& other) {
        this->type = other.type;
        this->health = other.health;
        this->damage = other.damage;
    }

    // Override Clone
```

```

IPrototype* clone() override {
    // Pass "this" to the copy constructor
    return new NPC(*this);
}

// Setters for Tweaking
void setHealth(int h) { health = h; }
void setType(std::string t) { type = t; }

void display() {
    std::cout << type << " [Health: " << health << ", Damage: " <<
damage << "]\n";
}
};

// Client usage
int main() {
    // 1. Expensive creation happens ONLY ONCE
    NPC* baseAlien = new NPC("Base Alien", 100, 10);

    // 2. Cheap, instant copy
    NPC* clone1 = static_cast<NPC*>(baseAlien->clone());

    // 3. Tweak the clone using setters (Customizing Clones)
    clone1->setType("Powerful Alien");
    clone1->setHealth(200);

    // Output results
    baseAlien->display(); // Output: Base Alien [Health: 100, Damage: 10]
    clone1->display();    // Output: Powerful Alien [Health: 200, Damage:
10]

    delete baseAlien;
    delete clone1;
    return 0;
}

```

8. CUSTOMIZING CLONES

As seen in the code skeleton, you do not need to build a new object from scratch just because you need a slight variation. You clone the base object (which gives you a perfect 100% replica) and then use standard setter methods (like `setHealth(200)`) to quickly tweak the properties of the cloned object, turning it into a variation instantly without running the heavy constructor again.

9. REAL-WORLD USE CASES

- **Video Game Entities:** Spawning hundreds of mostly identical enemies, bullets, or trees where running the constructor every time would cause frame drops.
- **Caching Complex Objects:** If an object requires heavy DB queries to build, you build it once, cache it, and serve clones of it to clients requesting that data.

10. INTERVIEW QUICK REFERENCE

Definition: It lets you create new objects by copying (or cloning) an existing instance, allowing the system to bypass the expensive standard constructor completely.

When to use:

- When object creation relies on heavy database connections, complex network calls, or intensive calculations.
- When you need to instantiate many small objects that share mostly similar states with very minor variations.

Commonly confused with: *Flyweight Pattern*.

While both aim to save memory/processing when creating many objects, Flyweight shares a single common object among many clients to reduce RAM, whereas Prototype creates physically distinct, independent copies (clones) of an object so they can be modified independently.

Key interview questions:

Q1: Why is the Marker Interface necessary?

A1: It guarantees to the client that any object implementing the interface safely supports the `clone()` operation.

Q2: Explain the danger of using a Shallow Copy when cloning.

A2: If the object holds pointers to other custom objects in the heap, a shallow copy only copies the memory address. This means the original and the clone will point to the exact same internal object, and altering it in the clone will unintentionally mutate the original.

Q3: How do you achieve a Deep Copy?

A3: Inside the copy constructor, you must explicitly write code that allocates new heap memory for the internal custom object, copying its values over, rather than just assigning the pointer.